

THE LAYOUT CATALOGUE

Every value, in one place. Keep this one open while you work — the lecture deck is the story, this is the lookup.

YOSSI ARVIV · FRONT-END WEB DEVELOPMENT · RUPPIN

display — WHAT KIND OF BOX

VALUE	LINE BEHAVIOUR	WIDTH / HEIGHT	VERTICAL MARGIN
<code>block</code>	starts a new line, takes the width offered	yes	yes
<code>inline</code>	flows in the line, content width	ignored	ignored
<code>inline-block</code>	flows in the line	yes	yes
<code>flex</code>	block-level container, children become flex items	yes	yes
<code>inline-flex</code>	inline-level container, children still flex items	yes	yes
<code>none</code>	no box at all	—	—

`inline-block` is an *inline* box, so the whitespace between two of them in the source is rendered as a space. That is the gap.

FOUR WAYS TO HIDE, AND WHAT EACH ONE COSTS

METHOD	KEEPS ITS SPACE	REACHABLE BY TAB	READ BY A SCREEN READER
<code>display: none</code>	no	no	no
<code>hidden</code> attribute	no	no	no
<code>visibility: hidden</code>	yes	no	no
<code>opacity: 0</code>	yes	yes	yes

`opacity: 0` is **only transparent**. It is still there, still focusable and still clickable — which is a bug that is very hard to see.

position — THE FIVE VALUES

VALUE	IN NORMAL FLOW	OFFSETS MEASURED FROM	ON SCROLL
<code>static</code>	yes	offsets ignored	scrolls
<code>relative</code>	yes — keeps its space	its own original position	scrolls
<code>absolute</code>	no	nearest positioned ancestor	scrolls
<code>fixed</code>	no	the viewport	stays put
<code>sticky</code>	yes — keeps its space	itself, then its scroll container	scrolls, then pins

"Out of flow" means nothing reserves its space any more: the siblings close the gap, and a parent with no other content collapses to nothing.

WHAT EACH BOX IS MEASURED AGAINST

- in normal flow** The nearest block-level ancestor's **content box**.
- absolute** The nearest ancestor with `position` other than `static` — its **padding box**.
- fixed** The viewport — *unless* an ancestor has a `transform`, `filter` or `perspective`, in which case that ancestor.
- sticky** Its nearest **scroll container**. No scroll container, no sticking.

Every percentage in an element — `width: 50%`, `left: 10%`, `padding: 5%` — resolves against this box.

Percentage heights are the exception that bites: they need the containing block to have a *definite* height. If it is `auto`, `height: 100%` resolves to `auto` too.

Use `min-height: 100dvh` instead. It asks nobody.

WHAT CREATES A STACKING CONTEXT

position

Not static, **and** a z-index other than auto

position

fixed or sticky — always, z-index or not

opacity

Anything below 1

transform · filter

Anything other than none

will-change

Naming any property that would create one

isolation

isolate — the only one you write on purpose

mix-blend-mode

Anything other than normal

Inside a stacking context, z-index values are compared **only with each other**. Against anything outside, what competes is the *context's own* z-index.

So a child with z-index: 999 inside a context whose own z-index is auto loses to a sibling of that context with z-index: 1.

Raising the number never helps. Find the ancestor and either remove what made it a context, or give it a z-index.

overflow

VALUE	CLIPPED	SCROLL CONTAINER	CONTENT STILL REACHABLE
<code>visible</code>	no — paints outside	no	yes
<code>hidden</code>	yes	yes	yes — Tab and Ctrl-F scroll to it
<code>scroll</code>	yes	yes, bars always shown	yes
<code>auto</code>	yes	yes, bars when needed	yes
<code>clip</code>	yes	no	no

Any scroll container becomes the box a `sticky` descendant sticks inside — which is the usual reason a correct `sticky` does nothing.

FLEXBOX · ON THE CONTAINER

display	<code>flex</code> · <code>inline-flex</code>
flex-direction	<code>row</code> · <code>row-reverse</code> · <code>column</code> · <code>column-reverse</code>
flex-wrap	<code>nowrap</code> · <code>wrap</code> · <code>wrap-reverse</code>
justify-content	<code>flex-start</code> · <code>flex-end</code> · <code>center</code> · <code>space-between</code> · <code>space-around</code> · <code>space-evenly</code> — main axis
align-items	<code>stretch</code> · <code>flex-start</code> · <code>flex-end</code> · <code>center</code> · <code>baseline</code> — cross axis
align-content	Same values, but for the <i>lines</i> . Only does anything with <code>wrap</code> .
gap	<code>row-gap</code> and <code>column-gap</code> . Space <i>between</i> items only, never outside.

WHICH AXIS IS WHICH

FLEX-DIRECTION	MAIN AXIS	CROSS AXIS
<code>row</code>	horizontal →	vertical ↓
<code>row-reverse</code>	horizontal ←	vertical ↓
<code>column</code>	vertical ↓	horizontal →
<code>column-reverse</code>	vertical ↑	horizontal →

`justify-content` is **always** the main axis.

`align-items` is **always** the cross axis.

Neither of them is "horizontal". Which direction they act in is decided by `flex-direction`, one line earlier.

Do not memorise "justify is horizontal". It is true in `row` and false in `column`, and it is the single most common Flexbox mistake.

FLEXBOX · ON AN ITEM

- flex-grow** Share of the *leftover* space this item takes. Default `0`.
- flex-shrink** Share of the *overflow* this item gives up. Default `1`.
- flex-basis** The size it starts from, along the main axis. Default `auto` = its content.
- align-self** Overrides `align-items` for this one item.
- order** Default `0`. Lower comes first. Negative allowed. **Visual only.**

Start with `flex-basis` to set the natural size, then tweak `flex-grow` and `flex-shrink`.

THE `flex` SHORTHAND

YOU WRITE	IT MEANS	USE IT WHEN
<code>flex: 1</code>	<code>1 1 0%</code>	equal widths, content ignored
<code>flex: auto</code>	<code>1 1 auto</code>	wider content stays wider
<code>flex: none</code>	<code>0 0 auto</code>	never grows or shrinks — a logo
<code>flex: 1 1 17rem</code>	as written	the responsive card row
<code>flex: 0 0 200px</code>	as written	a fixed sidebar

The free space is $\text{container} - \text{gaps} - \sum \text{basis}$, and each item gets $\text{basis} + (\text{free} \times \text{its grow} \div \sum \text{grow})$. When the free space is negative, `flex-shrink` takes over instead.

THE FIVE THAT WILL COST YOU TIME

1 · ABSOLUTE WITH NO ANCHOR

It aligned to the page, not to your box. `position: relative` is missing on the **ancestor**.

2 · Z-INDEX ACROSS CONTEXTS

999 lost to 1. An ancestor has `opacity`, `transform` or `filter`. Raising the number cannot help.

3 · JUSTIFY / ALIGN SWAPPED

You changed `flex-direction` and the axes went with it. Neither one is "horizontal".

4 · MARGIN: AUTO EVERYWHERE

It absorbs free space — that is all. Legitimate for centring a block, and for pushing one flex item to the end.

5 · HEIGHT: 100% DOING NOTHING

100% of `auto` is `auto`. Use `min-height: 100dvh`.

AND THE BONUS · MIN-WIDTH

A flex item will not shrink below its content. `min-width: 0` is the fix, and it is on the item.

WHERE TO LOOK THINGS UP

REFERENCE

MDN — search "mdn flexbox", "mdn position". The *Formal definition* table on each page tells you the default and whether it inherits.

PRACTICE

Flexbox Froggy — twenty-four levels, twenty minutes, free. Do it once and the property names stop needing to be looked up.

IN THE BROWSER

DevTools marks every flex container with a `flex` badge. Click it and the axes, the gaps and the free space are drawn on the page.

And the demos from today are yours: `demos/06-flex/playground/` answers most questions faster than a search does.